Project Part 2

Group 64
Aerin Brown (261076118)
Stalbek Ulanbek uulu (261102435)
Nehir Özsunar (261142760)

Views

View 1

a. The UpcomingPortCall view limits the Port_Calls table to just those that have yet to happen. This helps streamline queries regarding port calls that still need to be monitored or organized by excluding all port calls that are already done.

b.
CREATE VIEW UpcomingPortCall (voyage_id, sequence_number, arrival_date) AS (SELECT voyage_id, sequence_number, arrival_date FROM Port_Call WHERE arrival_date > CURRENT DATE)

c.

```
db2 => CREATE VIEW UpcomingPortCall (voyage_id, sequence_number, arrival_date) AS (SELECT voy
age_id, sequence_number, arrival_date FROM Port_Call WHERE arrival_date > CURRENT DATE)
DB20000I  The SQL command completed successfully.
```

d. SELECT * FROM UpcomingPortCall FETCH FIRST 5 ROWS ONLY

```
db2 => SELECT * FROM UpcomingPortCall FETCH FIRST 5 ROWS ONLY

VOYAGE_ID   SEQUENCE_NUMBER ARRIVAL_DATE
----------- --------------- ------------
          2               2 03/12/2026
          3               1 03/11/2026
          3               2 03/17/2026
          4               1 03/16/2026
          4               2 03/22/2026

  5 record(s) selected.
```

e. INSERT INTO UpcomingPortCall VALUES (1, 3, “2026-03-11”);

```
db2 => INSERT INTO UpcomingPortCall VALUES (1, 3, '2026-03-11')
DB20000I  The SQL command completed successfully.
```

f. The insert succeeded because UpcomingPortCall is defined over a single base table (Port_Call) with no aggregation, GROUP BY, or DISTINCT, making it an updatable view in DB2. Per the DB2 manual, inserts through such views are permitted as they can be directly mapped to the underlying table.

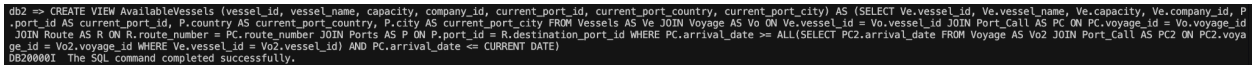
View 2

a. The AvailableVessels view is meant to aid with scheduling by showing vessels that are not currently scheduled for a voyage. It lists all vessels with no planned port calls alongside the port they are currently at, as determined by their last port call.

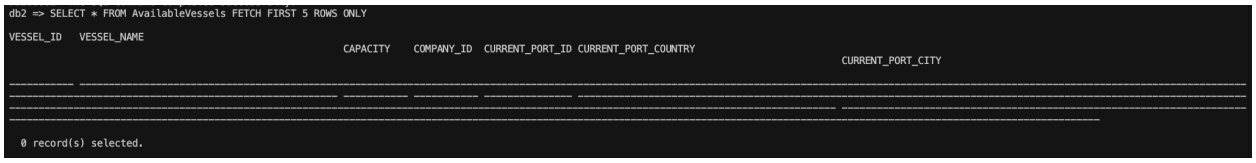
b.

CREATE VIEW AvailableVessels (vessel_id, vessel_name, capacity, company_id, current_port_id, current_port_country, current_port_city) AS (SELECT Ve.vessel_id, Ve.vessel_name, Ve.capacity, Ve.company_id, P.port_id AS current_port_id, P.country AS current_port_country, P.city AS current_port_city FROM Vessels AS Ve JOIN Voyage AS Vo ON Ve.vessel_id = Vo.vessel_id JOIN Port_Call AS PC ON PC.voyage_id = Vo.voyage_id JOIN Route AS R ON R.route_number = PC.route_number JOIN Ports AS P ON P.port_id = R.destination_port_id WHERE PC.arrival_date >= ALL(SELECT PC2.arrival_date FROM Voyage AS Vo2 JOIN Port_Call AS PC2 ON PC2.voyage_id = Vo2.voyage_id WHERE Ve.vessel_id = Vo2.vessel_id) AND PC.arrival_date <= CURRENT DATE)

c.



d.



SELECT * FROM AvailableVessels FETCH FIRST 5 ROWS ONLY

e.

INSERT INTO AvailableVessels VALUES (7, “MV Bluenose”, 6300, 5, 2, “Canada”, “Halifax”);

f. The insert failed with SQL0150N because AvailableVessels is defined over multiple joined tables with a subquery in the WHERE clause. DB2 does not permit insert, update, or delete operations on views that are not directly updatable due to joins or subqueries.

Checks

Check 1

a. The containertypes constraint limits the kinds of containers we can insert into the database to just dry, refrigerated, tank, and open top. This makes matching cargos to containers easy by standardizing naming conventions for various types of containers.

b. ALTER TABLE Container ADD CONSTRAINT containertypes CHECK (container_type IN (“Dry”, “Refrigerated”, “Tank”, “Open Top”));

c.

```
db2 => ALTER TABLE Container ADD CONSTRAINT containertypes CHECK (container_type IN ('Dry', 'Refrigerated', 'Tank', 'Open Top'))
DB20000I  The SQL command completed successfully.
```

d. INSERT INTO Container VALUES (1011, “Cooler”);

```
db2 => INSERT INTO Container VALUES (1011, 'Cooler')
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G64.CONTAINER.CONTAINERTYPES".
SQLSTATE=23513
```

Check 2

a. The positivecapacity constraint forces vessels to have a capacity greater than zero. All vessels need to have some real capacity to be functional as cargo vessels, so this is important to maintain.

b. ALTER TABLE Vessels ADD CONSTRAINT positivecapacity CHECK (capacity > 0);

c.

```
db2 => ALTER TABLE Vessels ADD CONSTRAINT positivecapacity CHECK (capacity > 0)
DB20000I  The SQL command completed successfully.
```

d. INSERT INTO Vessels VALUES (7, “MV Supernova”, 0, 5);

```
db2 => INSERT INTO Vessels VALUES (7, 'MV Supernova', 0, 5)
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G64.VESSELS.POSITIVECAPACITY".
SQLSTATE=23513
```

Relational Schema

1. Carrier_Companies(company_id, company_name)

2. Point_of_Contacts(person_id ,name, email, phone, company_id, party_id): company_id foreign key referencing relation Carrier_Companies, party_id foreign key referencing relation Shipping Party

3. Vessels(vessel_id, vessel_name, capacity, company_id): company_id foreign key referencing relation Carrier_Companies

4. Voyage(voyage_id, departure_date, vessel_id): vessel_id foreign key referencing relation Vessels.
5. Cargos(cargo_id, description, container_number): container_number foreign key referencing relation Container

6. Container(container_number, container_type)

7. Dangerous_Cargo(cargo_id, procedure_id): cargo_id foreign key referencing relation Cargos, procedure_id foreign key referencing relation Safety_Procedures

8. Standard_Cargo(cargo_id, weight, volume): cargo_id foreign key referencing relation Cargos

9. Safety_Procedure(procedure_id, procedure_description)

10. Route(route_number, origin_port_id, destination_port_id): origin_port_id and destination_port_id foreign key referencing relation Ports

11. Ports(port_id, country, city)

12. Port_Call(voyage_id, sequence_number, arrival_date, route_number): route_number foreign key referencing relation Route, voyage_id foreign key referencing relation Voyage

13. Shipping_Party(party_id, party_name)

14. Receives(party_id, voyage_id, sequence_number, cargo_id): party_id foreign key references relation Shipping_Party, voyage_id and sequence_number references relation Port_Call(voyage_id, sequence_number), cargo_id foreign key referencing relation Cargos

15. Sends(voyage_id, cargo_id, party_id): voyage_id foreign key referencing relation Voyage, cargo_id foreign key referencing relation Cargos, party_id foreign key referencing relation Shipping_Party

Pending Constraints

We couldn’t express in DB2 that a cargo must belong to either Standard_Cargo or Dangerous_Cargo but not both. We also can’t enforce that sequence_number values in Port_Call are strictly sequential per voyage.

5 QUERIES

QUERY 1: For each voyage carrying dangerous cargo, shows the vessel name, carrier company, and the required safety procedure.

SELECT V.voyage_id, Ve.vessel_name, CC.company_name, SP.procedure_description FROM Voyage AS V JOIN Vessels AS Ve ON V.vessel_id = Ve.vessel_id JOIN Carrier_Companies AS CC ON Ve.company_id = CC.company_id JOIN Sends AS S ON V.voyage_id = S.voyage_id JOIN Dangerous_Cargo AS DC ON S.cargo_id = DC.cargo_id JOIN Safety_Procedure AS SP ON DC.procedure_id = SP.procedure_id

```
db2 => SELECT V.voyage_id, Ve.vessel_name, CC.company_name, SP.procedure_description FROM Voyage AS V JOIN Vessels AS Ve ON V.vessel_id = Ve.vessel_id JOIN Carrier_Companies AS CC ON Ve.company_id = CC.company_i
d JOIN Sends AS S ON V.voyage_id = S.voyage_id JOIN Dangerous_Cargo AS DC ON S.cargo_id = DC.cargo_id JOIN Safety_Procedure AS SP ON DC.procedure_id = SP.procedure_id

VOYAGE_ID   VESSEL_NAME
                                                  COMPANY_NAME
                                                                                        PROCEDURE_DESCRIPTION

----------- --------------------------------------------------------------------------------
        1 MV Aurora 3000
                                                  OceanBridge Logistics
                                                                                        Toxic substance isolation

        3 MV Browne
                                                  BlueWave Marine
                                                                                        Flammable materials handling

        5 MV Sokuluk
                                                  Atlas Shipping
                                                                                        Corrosive material containment

  3 record(s) selected.
```

QUERY 2: For each carrier company, shows the total number of voyages made and the total vessel capacity deployed

SELECT CC.company_name, COUNT(DISTINCT V.voyage_id) AS total_voyages, SUM(Ve.capacity) AS total_capacity FROM Carrier_Companies AS CC JOIN Vessels AS Ve ON CC.company_id = Ve.company_id JOIN Voyage AS V ON Ve.vessel_id = V.vessel_id GROUP BY CC.company_name

```
db2 => SELECT CC.company_name, COUNT(DISTINCT V.voyage_id) AS total_voyages, SUM(Ve.capacity) AS total_capacity FROM Carrier_Companies AS CC JOIN Vessels AS Ve ON CC.company_id = Ve.company_id JOIN Voyage AS V O
N Ve.vessel_id = V.vessel_id GROUP BY CC.company_name

COMPANY_NAME
                                       TOTAL_VOYAGES TOTAL_CAPACITY
---------------------------------------- ------------- -------------
Atlas Shipping
                                                   1          6500
BlueWave Marine
                                                   1          5400
Horizon Freight
                                                   1          5800
NorthStar Cargo
                                                   1          7000
OceanBridge Logistics
                                                   2         11200

  5 record(s) selected.
```

QUERY 3: For each destination port visited more than once, shows the total number of arrivals and total cargo items received there.

SELECT P.city, P.country, COUNT(PC.sequence_number) AS total_arrivals, COUNT(R.cargo_id) AS total_cargos_received FROM Ports AS P JOIN Route AS Ro ON P.port_id = Ro.destination_port_id JOIN Port_Call AS PC ON Ro.route_number = PC.route_number LEFT JOIN Receives AS R ON PC.voyage_id = R.voyage_id AND PC.sequence_number = R.sequence_number GROUP BY P.port_id, P.city, P.country HAVING COUNT(PC.sequence_number) > 1

```
db2 => SELECT P.city, P.country, COUNT(PC.sequence_number) AS total_arrivals, COUNT(R.cargo_id) AS total_cargos_received FROM Ports AS P JOIN Route AS Ro ON P.port_id = Ro.destination_port_id JOIN Port_Call AS P
C ON Ro.route_number = PC.route_number LEFT JOIN Receives AS R ON PC.voyage_id = R.voyage_id AND PC.sequence_number = R.sequence_number GROUP BY P.port_id, P.city, P.country HAVING COUNT(PC.sequence_number) > 1

CITY
                                        COUNTRY
                                                                                TOTAL_ARRIVALS TOTAL_CARGOS_RECEIVED
---------------------------------------- ---------------------------------------- -------------- --------------------
Turkiye
                                        Istanbul
                                                                                             7                    4
Hamburg
                                        Germany
                                                                                             7                    6
Antwerp
                                        Belgium
                                                                                             3                    2

  3 record(s) selected.
```

QUERY 4: Lists all shipping parties that appear as both a sender and a receiver of cargo.

WITH Senders AS (SELECT DISTINCT party_id FROM Sends), Receivers AS (SELECT DISTINCT party_id FROM Receives) SELECT SP.party_name FROM Shipping_Party AS SP JOIN Senders AS Se ON SP.party_id = Se.party_id JOIN Receivers AS Re ON SP.party_id = Re.party_id

```
db2 => WITH Senders AS (SELECT DISTINCT party_id FROM Sends), Receivers AS (SELECT DISTINCT party_id FROM Receives) SELECT SP.party_name FROM Shipping_Party AS SP JOIN Senders AS Se ON SP.party_id = Se.party_id
JOIN Receivers AS Re ON SP.party_id = Re.party_id

PARTY_NAME
----------------------------------------
Nehir Imports
Aerin Retail
Stalbek Traders
NAS Supply
Aerien Chemicals
Nehir Turkish Foods

  6 record(s) selected.
```

QUERY 5: Lists voyages carrying only standard cargo (no dangerous cargo) whose total cargo weight exceeds 1000 kg, along with vessel name and total weight.

SELECT V.voyage_id, V.departure_date, Ve.vessel_name, SUM(SC.weight) AS total_weight FROM Voyage AS V JOIN Vessels AS Ve ON V.vessel_id = Ve.vessel_id JOIN Sends AS S ON V.voyage_id = S.voyage_id JOIN Standard_Cargo AS SC ON S.cargo_id = SC.cargo_id WHERE V.voyage_id NOT IN (SELECT S2.voyage_id FROM Sends AS S2 JOIN Dangerous_Cargo AS DC ON S2.cargo_id = DC.cargo_id) GROUP BY V.voyage_id, V.departure_date, Ve.vessel_name HAVING SUM(SC.weight) > 1000

```
db2 => SELECT V.voyage_id, V.departure_date, Ve.vessel_name, SUM(SC.weight) AS total_weight FROM Voyage AS V JOIN Vessels AS Ve ON V.vessel_id = Ve.vessel_id JOIN Sends AS S ON V.voyage_id = S.voyage_id JOIN Sta
ndard_Cargo AS SC ON S.cargo_id = SC.cargo_id WHERE V.voyage_id NOT IN (SELECT S2.voyage_id FROM Sends AS S2 JOIN Dangerous_Cargo AS DC ON S2.cargo_id = DC.cargo_id) GROUP BY V.voyage_id, V.departure_date, Ve.ve
ssel_name HAVING SUM(SC.weight) > 1000

VOYAGE_ID   DEPARTURE_DATE VESSEL_NAME
                                                                        TOTAL_WEIGHT
---------- -------------- ---------------------------------------- ---------------------------
         2 03/05/2026     MV Sultan Suleiman
                                                                                3900.00
         4 03/15/2026     MV Jusup
                                                                                1800.40
         6 03/25/2026     MV Bishkek
                                                                                2100.50

  3 record(s) selected.
```

2 MODIFICATIONS

MOD1: Increases the weight of all standard cargos being sent on voyages that have not yet departed by 10%, reflecting revised cargo manifests.

UPDATE Standard_Cargo SET weight = weight * 1.10 WHERE cargo_id IN (SELECT S.cargo_id FROM Sends AS S JOIN Voyage AS V ON S.voyage_id = V.voyage_id WHERE V.departure_date > CURRENT DATE)

```
db2 => UPDATE Standard_Cargo SET weight = weight * 1.10 WHERE car
go_id IN (SELECT S.cargo_id FROM Sends AS S JOIN Voyage AS V ON S
.voyage_id = V.voyage_id WHERE V.departure_date > CURRENT DATE)
DB20000I  The SQL command completed successfully.
```

MOD2: Deletes all past port calls where no cargo was received, cleaning up completed and irrelevant port call records.

DELETE FROM Port_Call WHERE arrival_date < CURRENT DATE AND (voyage_id, sequence_number) NOT IN (SELECT voyage_id, sequence_number FROM Receives)

```
db2 => DELETE FROM Port_Call WHERE arrival_date < CURRENT DATE AN
D (voyage_id, sequence_number) NOT IN (SELECT voyage_id, sequence
_number FROM Receives)
DB20000I  The SQL command completed successfully.
```

Creativity

a. Description: This query ranks all voyages by the number of cargo items they carry using the RANK() window function. Window functions are an advanced SQL feature that allow ranking and analytical operations over result sets without collapsing rows like GROUP BY does.

b. **SQL statement:** SELECT V.voyage_id, V.departure_date, Ve.vessel_name, COUNT(S.cargo_id) AS cargo_count, RANK() OVER (ORDER BY COUNT(S.cargo_id) DESC) AS cargo_rank FROM Voyage AS V JOIN Vessels AS Ve ON V.vessel_id = Ve.vessel_id JOIN Sends AS S ON V.voyage_id = S.voyage_id GROUP BY V.voyage_id, V.departure_date, Ve.vessel_name

c. **Screenshot:**

```
db2 => SELECT V.voyage_id, V.departure_date, Ve.vessel_name, COUNT(S.cargo_id) AS cargo_count, RANK() OVER (ORDER BY COUNT(S.cargo_id) DESC) AS cargo_rank FROM Voyage AS V JOIN Vessels AS Ve ON V.vessel_id =
 Ve.vessel_id JOIN Sends AS S ON V.voyage_id = S.voyage_id GROUP BY V.voyage_id, V.departure_date, Ve.vessel_name

VOYAGE_ID   DEPARTURE_DATE VESSEL_NAME
                                                                          CARGO_COUNT CARGO_RANK
----------- -------------- ------------------------------------------------------------
---------------------------------------------------------------------------- ----------- --------------------
          1 03/01/2026     MV Aurora 3000
                                                                                    2                    1
          2 03/05/2026     MV Sultan Suleiman
                                                                                    2                    1
          3 03/10/2026     MV Browne
                                                                                    2                    1
          4 03/15/2026     MV Jusup
                                                                                    2                    1
          5 03/20/2026     MV Sokuluk
                                                                                    2                    1
          6 03/25/2026     MV Bishkek
                                                                                    2                    1

  6 record(s) selected.
```

Team contribution

Stalbek created the database schema (createtbl.sql) and loaded the data (loaddata.sql). Aerin designed the views and check-constraints. Nehir wrote the five SQL queries, two data modifications, and ran the final commands on the server. We did the creativity part all together.